GroundGate: checking the facts behind an agent's action before it runs

Perceptron Hackathon 2026 · Theme 6, AI You Can Trust · Problem 9 · Team Neural Genesis (TM-1EFBCA97D27)

1. Problem and theme

We chose Problem 9, Theme 6 (AI You Can Trust). An agent can send a well-formed call such as *refund_order(ORD-1001, C1, 12000)* that is still wrong: the order does not exist, belongs to someone else, or was already refunded. Permission checks and schema validation never look at that, and a sent refund cannot be recalled. The hard part is evidence and consequences, not prompting.

2. Understanding of the challenge

Checking a fact sounds like a lookup; the difficulty is the evidence around it. A cache is incomplete, so a missing entry proves nothing; a cached entry may be stale; the source can time out; queries cost time; and a correct check can go stale before the write. The guardrail must choose to run, verify, escalate or refuse within a lookup budget, and the write path must enforce that choice.

3. Proposed approach

GroundGate sits between agent and tools: plain Python, SQLite as system of record, no model in the decision path.

- **Typed action.** Each tool (refund, cancel, update address) has a schema. We derive the *premises* from it, not from the agent's explanation. A refund rests on four: the order exists, belongs to this customer, is in a refundable status, and the amount fits what is left.
- **Evidence cascade.** Check the cache, then spend at most one read-only lookup, only when risk calls for it. Authoritative evidence outranks cache. A missing entry or unreachable source is *unverifiable*, never *contradicted*.
- **Decision.** Contradicted: block. Supported: allow. Unverifiable: allow if reversible (logged), escalate if irreversible, block once the escalation budget is spent.
- **Protected executor.** The only code that can write. It needs a single-use, expiring HMAC token bound to the exact action, then re-derives and rechecks the premises inside one *BEGIN IMMEDIATE* transaction with a version-conditional update.
- **Agent.** LangGraph with a local open-weight model (7–8B instruct via Ollama) calling tools through a JSON interface only.

4. Minimum viable scope

A checker for one class of action: three order tools verified against SQLite on four premises. *Catching a bad action:* fabricated ids, wrong-customer refunds and already-refunded orders are blocked while the valid refund goes through. Both run today (93 automated tests, 110 attack cases). The finals demo adds the live agent.

5. Stretch ambitions

Risk routing: routing between cache and lookup by risk tier exists; next is an optional careful tier for natural-language policy claims, allowed only to downgrade a verdict. **Red-team suite:** 110 cases exist; a teammate will add cases written to beat the gate.

6. Novelty and differentiation

GroundGate is not another permission layer. Progent and AgentSpec decide whether an agent may call a tool; we check the facts that make a permitted call correct. The contribution is a study of how **evidence quality, freshness, lookup budgets and state changes** affect safety and completion of business actions, with a write path that enforces the result.

We do not claim each mechanism is new: HMAC tokens, idempotency keys and transactional rechecks are standard, and most late-change protection comes from the recheck, which any system can have. The closest work, AMV-Guard, re-verifies premises per action under a cost trade-off; we add the commit-time barrier and an external-effects analysis, and report where we lose.

7. Evaluation plan

- **Scenarios, labels first.** Six categories (refunds, cancellations, wrong customer, already refunded, stale evidence, changes after the check) plus valid controls. Labels are written before any arm runs; "executed" means the database row changed.
- **Held-out set.** 30 dev cases tune thresholds. 51 held-out cases were written without opening the source, frozen by SHA-256 and run once. This set is pre-registered, not independent; a teammate's set is next.
- **Baselines.** No guard, closed-world cache, always-query, VOI-style budgeted policy, NOMOS-style rule gate (both our reimplementations), executor-only recheck.
- **Reporting.** Wilson and bootstrap 95% intervals; "harmful passed the gate" reported apart from "harmful executed".

Results so far. Held-out: GroundGate executes 0 of 31 harmful actions (95% CI 0–11%) and completes 18 of 20 valid ones; no guard executes 27, closed-world cache 6, always-query 6. Eight harmful cases pass the gate; only the recheck stops them. Synthetic: at 0.2 lookups per action, routing completes 10.6 more points of valid actions than always-query (CI 9.8–11.4) but loses on a fresh-but-wrong cache (2.5% vs 0.0% executed harm).

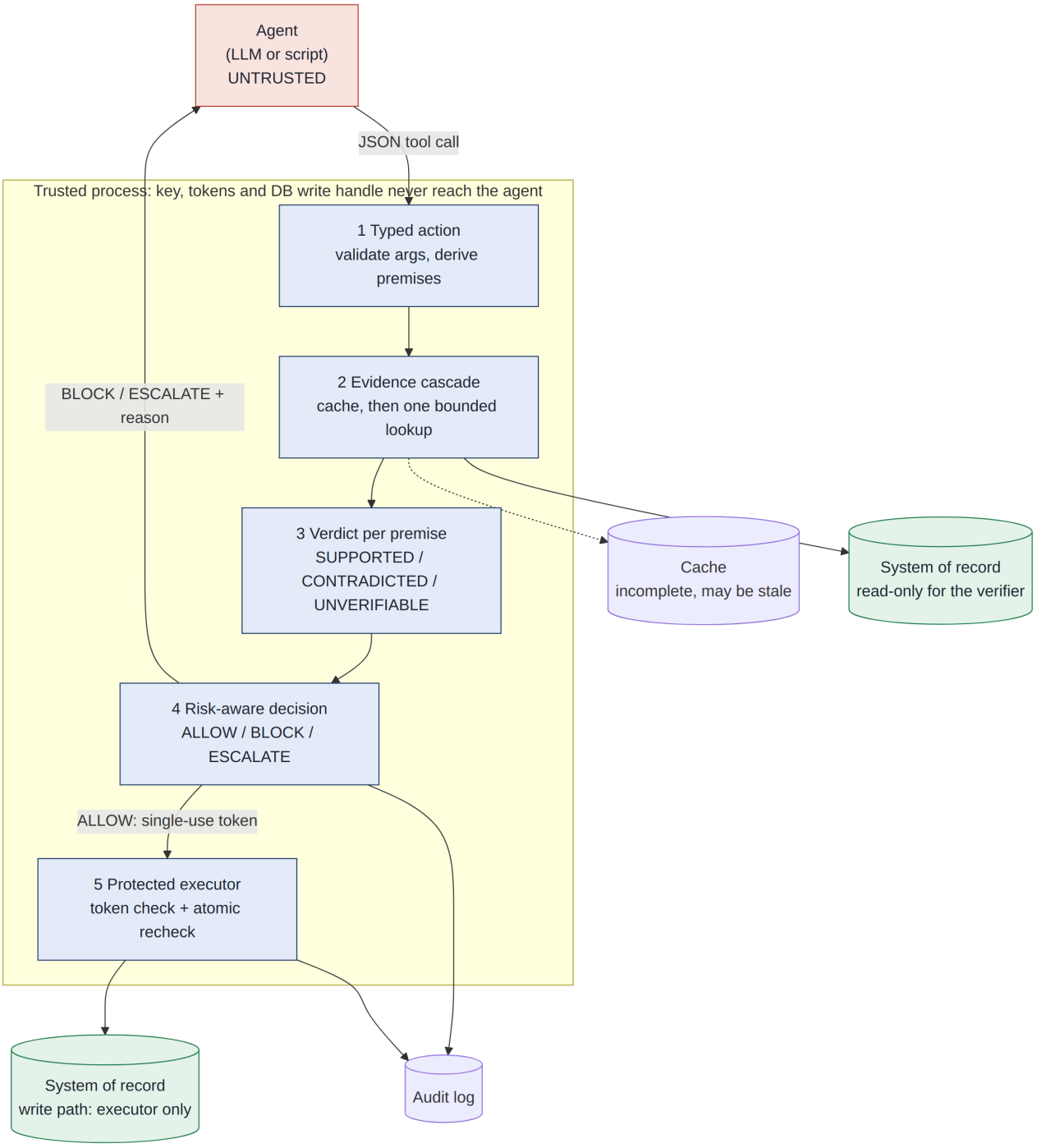
8. Feasibility and risks

Member	Skills	Owns
Dharmi Sandip Sapariya	DBMS, applied statistics, machine learning	Verification engine and protected executor; statistics; red-team and unit tests
Mouktika Jakkam	Python, LangGraph and local open-weight models, scenario design, writing	Agent integration (JSON interface only); held-out v2; documentation and demo

By	Milestone
15 Nov	Held-out v2 by a teammate, frozen by hash; run and failures listed by 20 Nov
26 Nov	LangGraph agent with local model: 120+ logged natural tool calls
1 Dec	Midway report: v2 results, agent traces
8 Dec	Outbox and reconciliation against a payment sandbox or mock ledger with injected timeouts; demo ready 14 Dec

Risks. Held-out v2 may expose failures: we report them. A local model may make few natural errors: we add provoking tasks and report how many were natural. SQLite cannot be atomic with an external API: mitigation (idempotency keys, reconciliation) is simulated so far. Costs and rates are assumed. The outbox is cut first if time runs short.

Architecture



The agent is untrusted. The key, tokens and write handle stay inside the trusted process; the verifier's database connection is read-only. Code, results and an interactive playground: github.com/DharmiSapariya/GroundGate